

TDS Archive



Image by author

★ Member-only story

GenAI with Python: RAG with LLM (Complete Tutorial)

Build your own ChatGPT with multimodal data and run it on your laptop without GPU



Mauro Di Pietro

Following ▾

12 min read · Jun 28, 2024



1K



17



In this article, I will show how to build a simple AI that can effectively learn knowledge from your personal documents and answer questions.



Photo by [Eric Krull](#) on [Unsplash](#)

Intro

Natural Language Processing (NLP) is the field of Artificial Intelligence that studies the interaction between machines and human language. The peak of NLP, so far, has been the arrival of **Large Language Models (LLM)**, trained on enormous amounts of text data, able to learn language patterns and variations. The term “Language Model” became common with the rise of Deep Learning and Neural Networks. In particular in 2018, when Google introduced the Transformer-based architecture, which significantly improved the performance of NLP models (Google’s BERT and OpenAI’s GPT).

Today, LLMs are typically used for **Text-to-Text** tasks or “Text Generation” (i.e. translation, summarization, chatbots and virtual assistants, or even writing entire books). Thanks to that, we have witnessed the rise of Generative Artificial Intelligence (GenAI), which is the field of the industry that focuses on creating new content (i.e. text, images, audio, video).

The state-of-the-art LLM landscape:

- ChatGPT by OpenAI, the most used (try it [here](#))
- Claude by Anthropic (try it [here](#))
- Gemini by Google (try it [here](#))
- Llama by Meta (try it [here](#))
- Phi by Microsoft, the smallest available model, so it can run on your laptop without a GPU (try it [here](#))
- StableLM by StabilityAI
- CommandR by Cohere (try it [here](#))
- Arctic by Snowflake (try it [here](#))
- Qwen by Alibaba (try it [here](#))
- Yi by 01AI
- Grok by X
- Megatron by NVIDIA
- Olympus by Amazon (not yet released)
- MM1 by Apple (not yet released)

ChatGPT is the most commonly used LLM, but companies have a problem with it: they can't upload their sensitive data on OpenAI (mostly for privacy and security). Therefore, companies are creating in-house AI services to leverage the power of LLMs on their private knowledge base. This kind of task is called **Retrieval-Augmented Generation (RAG)**. It's a technique that combines retrieval and generative models by adding to LLMs knowledge facts fetched from external sources.

In an organization, the knowledge base is usually composed of documents with multimodal content (i.e. text, images, spreadsheets), so the biggest challenge is processing them all in a way that machines can understand. In a nutshell, first every document is transformed into embeddings, then user queries are converted into the same vector space, allowing for a cosine similarity search.

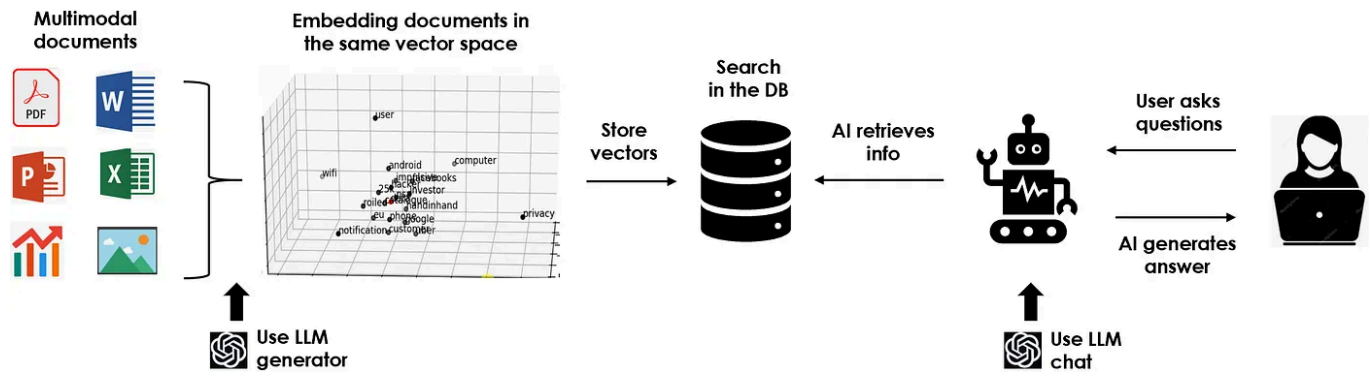


Image by author

In this tutorial, I'm going to create a RAG app using LLMs and multimodal data that can run on a normal laptop without GPU. I will present some useful Python code that can be easily applied in other similar cases (just copy, paste, run) and walk through every line of code with comments so that you can replicate this example (link to the full code below).

GitHub - mdipietro09/GenerativeAI: GenAI & LLM usecases and applications

GenAI & LLM usecases and applications. Contribute to mdipietro09/GenerativeAI development by creating an account...

[github.com](https://github.com/mdipietro09/GenerativeAI)

In particular, I will go through:

- **Setup:** import data with *Pdf2image*, OCR with *PyTesseract*.
- **Preprocessing:** use LLM with *Ollama* to enrich the data.
- **Database:** store and query data as vectors with *ChromaDB*.
- **Backend:** use LLM with *Ollama* to generate answers.
- **Frontend:** build an interface with *Streamlit* to interact with the AI.

Setup

In the real world, companies extensively use *PDFs* as most documents are converted into that format before sharing. Moreover, they represent a good use case because they contain images, tables, and text. Therefore, for this example, I will use the financial statement from a public company in *PDF* format as a dataset.

UNITED STATES SECURITIES AND EXCHANGE COMMISSION Washington, D.C. 20549		
FORM 10-K		
☒ ANNUAL REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934 For the fiscal year ended January 28, 2024		
OR		
☐ TRANSITION REPORT PURSUANT TO SECTION 13 OR 15(d) OF THE SECURITIES EXCHANGE ACT OF 1934 Commission file number: 0-23985		
 NVIDIA NVIDIA CORPORATION (Exact name of registrant as specified in its charter)		
Delaware (State or other jurisdiction of incorporation or organization)	94-3177549 (I.R.S. Employer Identification No.)	
2788 San Tomas Expressway, Santa Clara, California (Address of principal executive offices)	95051 (Zip Code)	
Registrant's telephone number, including area code: (408) 486-2000 Securities registered pursuant to Section 12(b) of the Act:		
Title of each class Common Stock, \$0.001 par value per share	Trading Symbol(s) NVDA	Name of each exchange on which registered The Nasdaq Global Select Market

[link](#)

There are 2 alternative ways to process a *PDF* document: read it as text or parse it as an image. Neither of them is perfect as it depends on the use case, but **OCR (Optical Character Recognition)** tends to work better, so I will do that.

Please note that the focus of this article isn't OCR, and I won't dedicate too much time to explaining how it works. If you want to deep dive into that topic, you can check out [this article](#) and play around with advanced OCR libraries like *[LayoutParser](#)* and *[Unstructured](#)*.

First of all, we need to convert the documents into images:

```
#conda install -c conda-forge poppler
import pdf2image #1.17.0

doc_img = pdf2image.convert_from_path("data/doc_nvidia.pdf", dpi=300)

# print one page as example
doc_img[35]
```

Fiscal Year 2024 Summary

	Year Ended		
	Jan 28, 2024	Jan 29, 2023	Change
(\$ in millions, except per share data)			
Revenue	\$ 60,922	\$ 26,974	Up 126%
Gross margin	72.7 %	56.9 %	Up 15.8 pts
Operating expenses	\$ 11,329	\$ 11,132	Up 2%
Operating income	\$ 32,972	\$ 4,224	Up 681%
Net income	\$ 29,760	\$ 4,368	Up 581%
Net income per diluted share	\$ 11.93	\$ 1.74	Up 586%

We specialize in markets where our computing platforms can provide tremendous acceleration for applications. These platforms incorporate processors, interconnects, software, algorithms, systems, and services to deliver unique value. Our platforms address four large markets where our expertise is critical: Data Center, Gaming, Professional Visualization, and Automotive.

Revenue for fiscal year 2024 was \$60.9 billion, up 126% from a year ago.

Data Center revenue for fiscal year 2024 was up 217%. Strong demand was driven by enterprise software and consumer internet applications, and multiple industry verticals including automotive, financial services, and healthcare. Customers across industry verticals access NVIDIA AI infrastructure both through the cloud and on-premises. Data Center compute revenue was up 244% in the fiscal year. Networking revenue was up 133% in the fiscal year.

Gaming revenue for fiscal year 2024 was up 15%. The increase reflects higher sell-in to partners following the normalization of channel inventory levels and growing demand.

Professional Visualization revenue for fiscal year 2024 was up 1%.

Automotive revenue for the fiscal year 2024 was up 21%. The increase primarily reflected growth in self-driving platforms.

Gross margin increased in fiscal year 2024, primarily driven by Data Center revenue growth and lower net inventory provisions as a percentage of revenue.

Operating expenses increased for fiscal year 2024, driven by growth in employees and compensation increases. Fiscal year 2023 also included a \$1.4 billion acquisition termination charge related to the proposed Arm transaction.

page 36

Then, we shall recognize text in the pages (images) using Tesseract, the main OCR system created by HP in 1985 and currently developed by Google.

```
import pytesseract #0.3.10

doc_txt = []
for page in doc_img:
    text = pytesseract.image_to_string(page)
    doc_txt.append(text)

doc_txt[35]
```

```
'Table of Contents\n\nFiscal Year 2024 Summary\n\nYear Ended\nJan 28, 2024 Jan 29, 2023 Change\n($ in millions, except per s  
hare data)\nRevenue $ 60,922 $ 26,974 Up 126%\nGross margin 72.7 % 56.9 % Up 15.8 pts\nOperating expenses $ 11,329 $ 11,132  
Up 2%\nOperating income $ 32,972 $ 4,224 Up 681%\nNet income $ 29,760 $ 4,368 Up 581%\nNet income per diluted share $ 11.93  
$ 1.74 Up 586%\n\nWe specialize in markets where our computing platforms can provide tremendous acceleration for application  
s. These platforms incorporate processors, ninterconnects, software, algorithms, systems, and services to deliver unique val  
ue. Our platforms address four large markets where our expertise is critical: Data nCenter, Gaming, Professional Visualizati  
on, and Automotive.\n\nRevenue for fiscal year 2024 was $60.9 billion, up 126% from a year ago.\n\nData Center revenue for f  
iscal year 2024 was up 217%. Strong demand was driven by enterprise software and consumer internet applications, and multipl  
e nindustry verticals including automotive, financial services, and healthcare. Customers across industry verticals access N  
VIDIA AI infrastructure both through the ncloud and on-premises. Data Center compute revenue was up 244% in the fiscal year.  
Networking revenue was up 133% in the fiscal year.\n\nGaming revenue for fiscal year 2024 was up 15%. The increase reflects  
higher sell-in to partners following the normalization of channel inventory levels and ngrowing demand.\n\nProfessional Visu  
alization revenue for fiscal year 2024 was up 1%. nAutomotive revenue for the fiscal year 2024 was up 21%. The increase prim  
arily reflected growth in self-driving platforms.\n\nGross margin increased in fiscal year 2024, primarily driven by Data Cent  
er revenue growth and lower net inventory provisions as a percentage of revenue.\n\nOperating expenses increased for fiscal  
year 2024, driven by growth in employees and compensation increases. Fiscal year 2023 also included a $1.4 billion nacquisit
```

page 36 parsed

Now that we have the raw text, we must preprocess it before anything else. We're going to use language models. Currently, there are several alternative libraries to leverage the power of LLMs: Ollama, HuggingFace, DSPy, LangChain. I like to use *Ollama* as it's both powerful and user-friendly.

First of all, you need to download the program from the website. Then, on the prompt, write the following command to run a LLM on your laptop (I'm using *Phi3* and *LLaVa*):

[Open in app](#) ↗

Medium



Search



Write



Model	Parameters	Size	Download
Llama 3	8B	4.7GB	<code>ollama run llama3</code>
Llama 3	70B	40GB	<code>ollama run llama3:70b</code>
Phi 3 Mini	3.8B	2.3GB	<code>ollama run phi3</code>
Phi 3 Medium	14B	7.9GB	<code>ollama run phi3:medium</code>
Gemma	2B	1.4GB	<code>ollama run gemma:2b</code>
Gemma	7B	4.8GB	<code>ollama run gemma:7b</code>
Mistral	7B	4.1GB	<code>ollama run mistral</code>
Moondream 2	1.4B	829MB	<code>ollama run moondream</code>
Neural Chat	7B	4.1GB	<code>ollama run neural-chat</code>
Starling	7B	4.1GB	<code>ollama run starling-lm</code>
Code Llama	7B	3.8GB	<code>ollama run codellama</code>
Llama 2 Uncensored	7B	3.8GB	<code>ollama run llama2-uncensored</code>
LLaVA	7B	4.5GB	<code>ollama run llava</code>
Solar	10.7B	6.1GB	<code>ollama run solar</code>

[Ollama GitHub](#)

At last, install the Python library (*`pip install ollama`*) and you're good to go.

Preprocessing

Just like in any Data Science project, the hardest part is always the dataset. No matter how much Machine Learning you put into it, if the data are not good, the AI won't work properly.

Regarding LLMs, since there are embeddings involved, the standard practice is to convert everything into text. I will show how to process the most common types of multimodal content (text, tables, images, plots).

TEXT — Since documents may have different structures and layouts, the dataset should be **split into paragraphs**. Besides raw text, one could enrich

the data using additional information (i.e. title, tags, links). This is called **metadata enrichment**.

```
# Using the Table of Contents to tag the paragraphs
title_map = {
    "4-12": "Business",
    "13-33": "Risk Factors",
    "34-44": "Financials",
    "45-46": "Directors",
    "47-83": "Data"
}

lst_docs, lst_ids, lst_metadata = [], [], []
for n, page in enumerate(doc_txt):
    try:
        ## get title
        title = [v for k, v in title_map.items()
                  if n in range(int(k.split("-")[0]),
                               int(k.split("-")[1])+1)][0]

        ## clean page
        page = page.replace("Table of Contents", "")
        ## get paragraph
        for i, p in enumerate(page.split('\n\n')):
            if len(p.strip()) > 5: ##<--clean paragraph
                lst_docs.append(p.strip())
                lst_ids.append(str(n) + "_" + str(i))
                lst_metadata.append({"title": title})
    except:
        continue

## print example
for id, doc, meta in zip(lst_ids[375:378],
                        lst_docs[375:378],
                        lst_metadata[375:378]):
    print(id, "-", meta, "\n", doc, "\n")
```



```

35_1 - {'title': 'Financials'}
Fiscal Year 2024 Summary

35_2 - {'title': 'Financials'}
Year Ended
Jan 28, 2024 Jan 29, 2023 Change
($ in millions, except per share data)
Revenue $ 60,922 $ 26,974 Up 126%
Gross margin 72.7 % 56.9 % Up 15.8 pts
Operating expenses $ 11,329 $ 11,132 Up 2%
Operating income $ 32,972 $ 4,224 Up 681%
Net income $ 29,760 $ 4,368 Up 581%
Net income per diluted share $ 11.93 $ 1.74 Up 586%

35_3 - {'title': 'Financials'}
We specialize in markets where our computing platforms can provide tremendous acceleration for applications. These platforms
incorporate processors,
interconnects, software, algorithms, systems, and services to deliver unique value. Our platforms address four large markets
where our expertise is critical: Data
Center, Gaming, Professional Visualization, and Automotive.

```

Image by author

Metadata enrichment with language models can significantly enhance document retrieval. For instance, I'm going to use *Phi3* to summarize each paragraph into a few keywords.

```

import ollama #0.2.1

def keyword_generator(p, top=3):
    prompt = "summarize the following paragraph in 3 keywords separated by ,: "+
    res = ollama.generate(model="phi3", prompt=prompt)["response"]
    return res.replace("\n", " ").strip()

## test
p = '''Professional artists, architects and designers use NVIDIA partner product
use cases, such as creating visual effects in movies or designing buildings and
class GPUs, as more enterprise customers develop and deploy AI applications with
print(keyword_generator(p))

```

Returns: “*Creative Design, NVIDIA Partnerships, AI Expansion*”. I will apply it to all the paragraphs:

```

from tqdm.notebook import tqdm

for i, doc in tqdm(enumerate(lst_docs)):
    lst_metadata[i]["keywords"] = keyword_generator(doc)

## print example
for id, doc, meta in zip(lst_ids[375:378],
                        lst_docs[375:378],
                        lst_metadata[375:378]):
    print(id, "-", meta, "\n", doc, "\n")

```

```

35_1 - {'title': 'Financials', 'keywords': 'Budget Allocation, Government Expenditure, Financial Planning'}
Fiscal Year 2024 Summary

35_2 - {'title': 'Financials', 'keywords': 'Revenue Growth, Margin Increase, Profit Surge'}
Year Ended
Jan 28, 2024 Jan 29, 2023 Change
($ in millions, except per share data)
Revenue $ 60,922 $ 26,974 Up 126%
Gross margin 72.7 % 56.9 % Up 15.8 pts
Operating expenses $ 11,329 $ 11,132 Up 2%
Operating income $ 32,972 $ 4,224 Up 681%
Net income $ 29,760 $ 4,368 Up 581%
Net income per diluted share $ 11.93 $ 1.74 Up 586%

35_3 - {'title': 'Financials', 'keywords': 'Computing Acceleration, Market Specialization, Diverse Platforms'}
We specialize in markets where our computing platforms can provide tremendous acceleration for applications. These platforms
incorporate processors,
interconnects, software, algorithms, systems, and services to deliver unique value. Our platforms address four large markets
where our expertise is critical: Data
Center, Gaming, Professional Visualization, and Automotive.

```

Image by author

TABLES — It's well known that LLMs can process sequential text with remarkable accuracy, however they've started understanding structured data as well, therefore tables can be **passed as normal text**. Let's double-check:

```

table = lst_docs[376]
print("Table:\n", table)

prompt = f"Summarize the following table: {table}"
res = ollama.generate(model="phi3", prompt=prompt)["response"]

print("\nSummary:\n", res)

```


	Year Ended		
	Jan 28, 2024	Jan 29, 2023	Change
	(\$ in millions, except per share data)		
Revenue	\$ 60,922	\$ 26,974	Up 126%
Gross margin	72.7 %	56.9 %	Up 15.8 pts
Operating expenses	\$ 11,329	\$ 11,132	Up 2%
Operating income	\$ 32,972	\$ 4,224	Up 681%
Net income	\$ 29,760	\$ 4,368	Up 581%
Net income per diluted share	\$ 11.93	\$ 1.74	Up 586%

Table:

Year Ended	Jan 28, 2024	Jan 29, 2023	Change
(\$ in millions, except per share data)			
Revenue	\$ 60,922	\$ 26,974	Up 126%
Gross margin	72.7 %	56.9 %	Up 15.8 pts
Operating expenses	\$ 11,329	\$ 11,132	Up 2%
Operating income	\$ 32,972	\$ 4,224	Up 681%
Net income	\$ 29,760	\$ 4,368	Up 581%
Net income per diluted share	\$ 11.93	\$ 1.74	Up 586%

Summary:

The table shows financial performance for the fiscal year ended January 28, 2024, compared to the same period in 2023:

- Revenue increased significantly by 126%, from \$26,974 million to \$60,922 million.
- Gross margin rose by 15.8 percentage points, reaching 72.7% compared to 56.9%.
- Operating expenses grew by a modest 2%, going up from \$11,132 million to \$11,329 million.
- Operating income surged by an impressive 681%, increasing from \$4,224 million in 2023 to \$32,972 million for 2024.
- Net income soared by 581%, jumping from \$4,368 million in 2023 to \$29,760 million in 2024.
- Net income per diluted share saw a substantial increase of 586%, going up from \$1.74 to \$11.93.

Image by author

IMAGES — The embedding of images can be quite tricky (involves reducing the dimensionality of a 3D array to a vector). Therefore, a common practice is to use AI to **generate a caption** describing the images, where possible, and treat the information as text.

```
from matplotlib import image, pyplot

image_file = "data/image.jpeg"

pyplot.imshow(image.imread(image_file))
pyplot.show()
```



Photo by [Andrey Matveev](#) on [Unsplash](#)

To be processed by LLMs, the image must be converted into a string:

```
## Encode the image into a base64 string
import base64

def encode_image(path):
    with open(path, "rb") as file:
        return base64.b64encode(file.read()).decode('utf-8')

img = encode_image(image_file)
img[:1000]
```

We can pass the string to a model asking to describe the image. Unfortunately, *Phi3* is not up to the task because it's not a vision model:

Hence, we need to get a proper **vision model**: Microsoft's LLaVa is an efficient choice as it can run also without a GPU (try it [here](#)).

15/31

Image by author

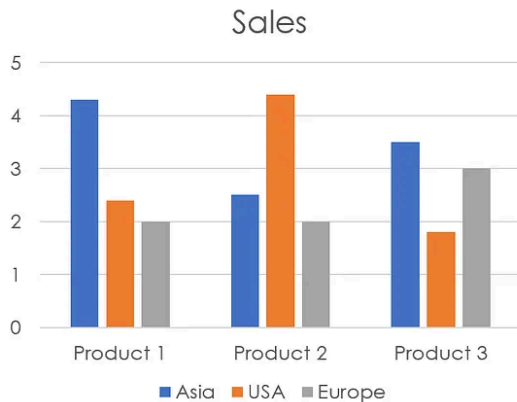
PLOTS — Same as for the images, vision models could read and summarize plots/charts into text, but the result is not as accurate. Let's see an example:

```
image_file = "data/plot.png"

pyplot.imshow(image.imread(image_file))
pyplot.show()

img = encode_image(image_file)

prompt = "Describe the image in detail. Be specific about graphs, such as bar plots"
res = ollama.generate(model="llava", prompt=prompt, images=[img])["response"]
print(res)
```



The image is a digital screenshot of a business performance dashboard. It features multiple charts and graphs that provide an overview of sales data for three products: "Product 1", "Product 2", and "Product 3". Below these graphs are two horizontal bar plots, presumably representing sales volumes or quantities across different regions: "America", "Africa", and "Europe".

The upper part of the image displays two line graphs with bar plots on top. The left graph represents "Asia" against a scale from 0 to 6, while the right graph shows "Europe" against the same scale.

Both graphs have vertical bars in blue for "Product 1", orange for "Product 2", and gray for "Product 3". The bars for each product are colored differently: blue for "Product 1", orange for "Product 2", and gray for "Product 3". The bars have a consistent height, suggesting that the sales volume in these regions is equal. ❌

On the left side of the image, there's a vertical bar chart with the title "Sales" at the top. This chart shows a count of sales activity ranging from 0 to 7. The vertical bars are colored according to the products: blue for "Product 1", orange for "Product 2", and gray for "Product 3". ❌

Image by author

Database

We can store our data in a vector database, which is the best solution for efficiently storing, indexing, and searching unstructured data. Nowadays, the most used vector db is Microsoft's AI Search, while the best open-source tool is ChromaDB, which is useful, easy, and free.

After a quick “*pip install chromadb*”, you can interact with the db using Python in three different ways:

- *chromadb.Client()* to create a db that stays temporarily in memory without occupying physical space on disk.
- *chromadb.PersistentClient(path)* to save and load the db from your local machine.
- *chromadb.HttpClient(host='localhost', port=8000)* to have a client-server mode on your browser.

When storing documents in *ChromaDB*, data are saved as vectors so that one can search with a query-vector to retrieve the closest matching records. Please note that, if not specified otherwise, the default embedding function is a sentence transformer model (*all-MiniLM-L6-v2*).

```
import chromadb #0.5.0

db = chromadb.PersistentClient()

collection_name = "nvidia"
collection = db.get_or_create_collection(name=collection_name,
                                         embedding_function=chromadb.utils.embedding_functions.DefaultEmbedder)

collection.add(documents=lst_docs, ids=lst_ids, metadatas=lst_metadata,
              images=None, embeddings=None)
collection.peek(1)
```

```
-0.06092095375061035,
-0.01741098240017891,
0.0484163761138916,
-0.09500133991241455,
0.03292495757341385,
0.021035268902778625]],
'metadatas': [{'keywords': 'Renewable Energy Adoption, Supplier Engagement, Emission Reduction Goals',
'title': 'Business'}],
'documents': ['or generate enough renewable energy to match 100% of our global electricity usage for our offices and data c
enters. In fiscal year 2023, we increased the\percentage of our total electricity use matched by renewable energy purchases
to 44%. By fiscal year 2026, we aim to engage manufacturing suppliers\ncomprising at least 67% of NVIDIA's scope 3 category
1 GHG emissions with goal of effecting supplier adoption of science-based targets.'],
'uris': None,
'data': None}
```

documents are saved as a dictionary with vector and metadata

Let's try to query something:

```
query = "how much is the revenue?"
collection.query(query_texts=[query])
```

```
{'ids': [['78_13',
'78_7',
'54_11',
'54_13',
'37_1',
'39_2',
'35_4',
'78_12',
'69_5',
'70_1']],
'distances': [[0.3881477812214862,
0.414599754463739,
0.6890628337860107,
0.7084571123123169,
0.7561033964157104,
0.7571808099746704,
0.7699214816093445,
0.7703176357219828,
0.8317533731460571,
0.8592502474784851]],
```

Image by author

The query has been embedded in the same vector space and *ChromaDB* retrieved the closest documents. We can join them and get the final response from the db:

```
res_db = collection.query(query_texts=[query])["documents"][0][0:10]
context = ' '.join(res_db).replace("\n", " ")
context
```

```
'Total revenue Total revenue $ 60,922 $ 26,974 $ 26,914 Revenue Recognition Product Sales Revenue Revenue Recognition Revenue
e Allowances Concentration of Revenue Revenue for fiscal year 2024 was $60.9 billion, up 126% from a year ago. Revenue: (In
millions) Data Center 47,525 15,005 $ 10,613 Gaming 10,447 9,067 12,462 Professional Visualization 1,553 1,544 2,111 Automot
ive 1,091 903 566 OEM and Other 306 455 1,162 60,922 26,974 $ 26,914 (2) Deferred revenue primarily includes customer advanc
es and deferrals related to support for hardware and software, license and development arrangements, and cloud services. $23
3 million and $35 million of the balance in fiscal 2024 and 2023 respectively, related to customer advances. Invoiced in fut
ure periods. Revenue allocated to remaining performance obligations, which includes deferred revenue and amounts that will b
e invoiced and recognized as revenue in future periods, was $1.1 billion as of January 28, 2024. We expect to recognize appr
oximately 40% of this revenue over the next twelve months and the remainder thereafter. This excludes revenue related to per
formance obligations for contracts with a length of one year or less.'
```

Image by author

The db response can be used as the **context**, a small set of information that the LLM can store and access thanks to its memory module. Using the prompt, we could specify whether the model must use exclusively the information in the context or also its knowledge learned during the training.

```
# Chat with Context only
res = ollama.chat(model="phi3",
                  messages=[{"role": "system", "content": "Give the most accurate
                           the folling information"},
                           {"role": "user", "content": query}])
print(res["message"]["content"])
```

```
The total recognized revenue for fiscal year 2024 was $60.9 billion, which represents an increase of 126% from the previous
year.
```

```
The breakdown by category in millions of dollars is as follows:
```

- Data Center: \$47,525 million
- Gaming: \$10,447 million
- Professional Visualization: \$1,553 million
- Automotive: \$1,091 million
- OEM and Other: \$306 million

```
The total reported revenue (in millions) of $60,922 includes $26,974 million for fiscal year 2024 and $26,914 million for the
previous year.
```

Image by author

Or adding the model's knowledge:

```
# Chat with knowledge + Context
res = ollama.chat(model="phi3",
                  messages=[{"role": "system", "content": "Give the most accurate
                                                                    and the folling additio
                                                                    {"role": "user", "content": query}]]
print(res["message"]["content"])
```

The total recognized revenue for fiscal year 2024 was \$60,922 million (or \$60.9 billion when expressed in billions). This amount represents an increase of 126% from the previous year's revenue. Additionally, there is a breakdown by product categories as follows:

- Data Center: \$47,525 million
- Gaming: \$10,447 million
- Professional Visualization: \$1,553 million
- Automotive: \$1,091 million
- OEM and Other: \$306 million

It's also important to note that there are deferred revenue amounts of \$233 million in fiscal 2024 and \$35 million in fiscal 2023 related to customer advances, which will be recognized as revenue over future periods. The remaining performance obligations account for approximately \$1.1 billion, with an expectation that about 40% of this amount will be recognized within the next twelve months.

Image by author

The database is set up and working perfectly. We can proceed to the next step and build the chatbot app to perform RAG, or, in simpler words, to ask questions about our documents.

Frontend

Nowadays, the most used Python library to build quick Web Apps is **Streamlit** as it simplifies the development of NLP applications with its streaming functionalities.

Let's start by defining the layout: I want my screen to have a sidebar where the user can see the chat history.

```
import streamlit as st #1.35.0

## Layout
st.title('💬 Write your questions')
st.sidebar.title("Chat History")
app = st.session_state

if "messages" not in app:
    app["messages"] = [{"role": "assistant", "content": "I'm ready to retrieve inf"}

if 'history' not in app:
    app['history'] = []

if 'full_response' not in app:
    app['full_response'] = ''
```

The *app* object (or *session_state*) is a dictionary with this kind of structure:

```
{'history': ['😄: how much is the revenue?',
             '🤖: The total revenue reported in the given information is 60million'],
 'messages': [{ 'role': 'assistant', 'content': 'I'm ready to retrieve information'},
               { 'role': 'user', 'content': 'how much is the revenue?'},
               { 'role': 'assistant', 'content': 'The total revenue reported in the given information is 60million'}],
 'full_response': 'The total revenue reported in the given information is 60million'}
```

example of a session_state

If you don't want disappearing messages in the chat, add the following code:

```
## Keep messages in the Chat
for msg in app["messages"]:
    if msg["role"] == "user":
        st.chat_message(msg["role"], avatar="😄 ").write(msg["content"])
    elif msg["role"] == "assistant":
        st.chat_message(msg["role"], avatar="🤖 ").write(msg["content"])
```

Now, we must code the chat core: the user's questions shall be added to the messages list, passed to the AI, and the app will **stream the response**.

```
## Chat
if txt := st.chat_input():
    ### User writes
    app["messages"].append({"role": "user", "content": txt})
    st.chat_message("user", avatar="😎").write(txt)

    ### AI responds with chat stream
    app["full_response"] = ""
    st.chat_message("assistant", avatar="🤖").write_stream( xxx )
    app["messages"].append({"role": "assistant", "content": app["full_response"]})

    ### Show sidebar history
    app['history'].append("😎: "+txt)
    app['history'].append("🤖: "+app["full_response"])
    st.sidebar.markdown("<br />".join(app['history'])+"<br /><br />", unsafe_all
```

I inserted a placeholder (“xxx”) that we will substitute with the function to respond to user questions, right after we finish coding it. When running, the app looks like this:

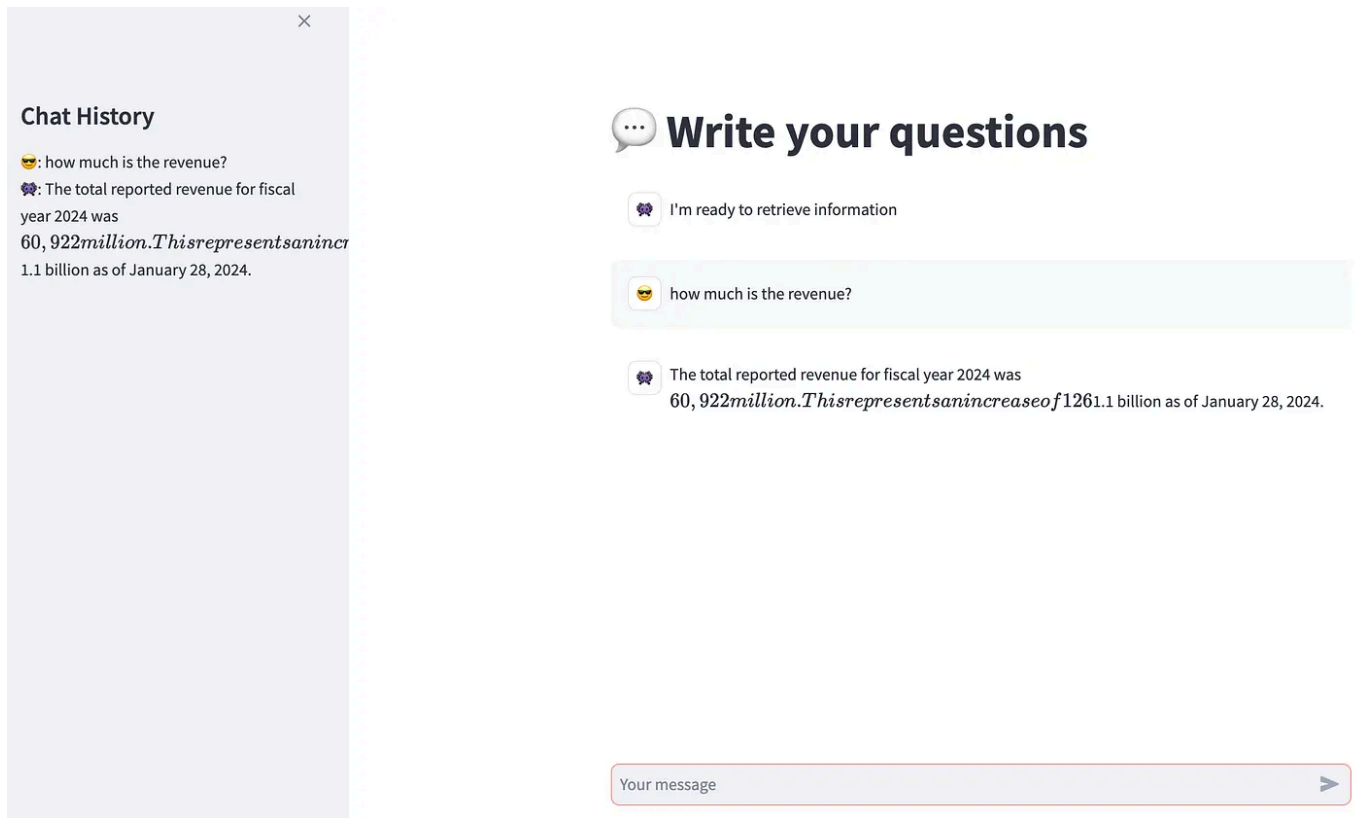


Image by author

Backend

We are already familiar with this part, it just has to be put inside a class. There are some points to highlight:

- By calling the collection name in *ChromaDB*, if it already exists the db will “get” and not “create” a new one.
- The streaming parameter of the *Ollama* chat function should be switched on (*stream=True*) to return chunks of text instead of the full response.
- I’m passing the whole chat history as input every time I call the LLM so that it can understand references to previous messages.

```
import chromadb #0.5.0
import ollama #0.5.0
```

```
class AI():
    def __init__(self):
        db = chromadb.PersistentClient()
        self.collection = db.get_or_create_collection("nvidia")

    def query(self, q, top=10):
        res_db = self.collection.query(query_texts=[q])["documents"][0][0:top]
        context = ' '.join(res_db).replace("\n", " ")
        return context

    def respond(self, lst_messages, model="phi3", use_knowledge=False):
        q = lst_messages[-1]["content"]
        context = self.query(q)

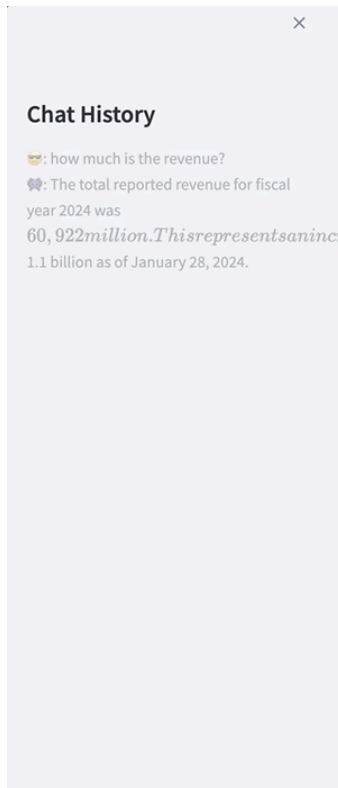
        if use_knowledge:
            prompt = "Give the most accurate answer using your knowledge and the folling"
        else:
            prompt = "Give the most accurate answer using only the folling information: \

        res_ai = ollama.chat(model=model,
                               messages=[{"role": "system", "content": prompt}] + lst_messages,
                               stream=True)

        for res in res_ai:
            chunk = res["message"]["content"]
            app["full_response"] += chunk
            yield chunk

ai = AI()
```

To run a *Streamlit* app, you must use this command: *streamlit run rag_app.py*



Write your questions

I'm ready to retrieve information

how much is the revenue?

The total reported revenue for fiscal year 2024 was 60,922million.This represents an increase of 1261.1 billion as of January 28, 2024.

how did they make it?

Your message

You can see the full code on [my GitHub](#)

Conclusion

This article has been a tutorial to demonstrate **how to build a RAG app with Python and LLMs**. We processed multimodal contents using language models and stored the data in a vector database that a ChatGPT-like AI uses when responding to our questions. Finally, we built a user-friendly interface to chat with the model.

I hope you enjoyed it! Feel free to contact me for questions and feedback or just to share your interesting projects.

👉 Let's Connect 👉

*This article is part of the series **GenAI with Python**, see also:*